

Spesifikasi “Tugas Besar”

IF1221 Logika Komputasional

Tahun 2025/2026

Astro-Scheduler: Optimasi Penjadwalan Ekspedisi

Dipersiapkan Oleh Bernhard Aprillio Pramana / 13524074

Waktu Mulai: xx

Waktu Selesai: xx

Topik

Membuat sebuah program purwarupa (*proof of concept*) berbasis CLI (atau GUI sederhana untuk bonus) menggunakan bahasa pemrograman Python untuk menyelesaikan *Constraint Satisfaction Problem* (CSP) melalui pendekatan algoritma Local Search. Implementasi tugas besar harus mengandung algoritma:

1. Hill-Climbing (Seluruh Varian: *Basic*, *Sideways Move*, *Stochastic*, *Random Restart*)

2. Simulated Annealing
3. Genetic Algorithm

Tujuan

Tujuan dari tugas besar ini adalah mengimplementasikan dan membandingkan performa dari berbagai algoritma Local Search dalam menyelesaikan masalah kombinatorial yang kompleks. Mahasiswa diharapkan mampu menerjemahkan masalah dunia nyata ke dalam representasi state, mendefinisikan *neighbor/successor function*, merancang fungsi heuristik, serta **memvisualisasikan** proses pencarian solusi untuk kemudahan analisis performa antar-algoritma.

Deskripsi Permasalahan

Dalam era penjelajahan galaksi, *Astral Express* perlu menjadwalkan N misi ekspedisi ke M planet dalam T hari. Kamu bertugas sebagai komandan navigator yang harus mendistribusikan misi-misi tersebut ke berbagai armada kapal. Penjadwalan ini penuh dengan konflik (bentrok armada, ketersediaan planet, dll). Karena mencari solusi *exact* membutuhkan waktu komputasi yang terlalu lama, penyelesaian dilakukan dengan pendekatan algoritma Local Search yang bergerak dari konfigurasi awal secara iteratif untuk meminimalkan bentrok (pelanggaran jadwal) hingga mencapai jadwal yang sempurna atau batas waktu komputasi habis.

Representasi State dan Batasan

- **Representasi State:** Menggunakan *complete-state formulation* di mana seluruh misi ekspedisi telah dialokasikan ke armada tertentu, pada planet tertentu, dan pada hari tertentu secara lengkap sejak awal pencarian.
- **Formulasi Initial State:** Jadwal pertama kali dibangun secara acak (*random*). Setiap misi akan dialokasikan ke armada, planet, dan hari

secara acak, yang dipastikan akan menghasilkan nilai penalti (konflik) awal yang tinggi.

- **Hard Constraint 1:** Satu armada tidak diperbolehkan menjalankan lebih dari satu misi pada hari yang sama.
- **Hard Constraint 2:** Satu planet hanya diperbolehkan dikunjungi oleh satu armada pada hari yang sama.

Successor Function dan Objective Function

- **Langkah Neighbor (Shift):** Memilih 1 misi secara acak dari state saat ini, lalu mengubah alokasi harinya atau armadanya ke opsi lain yang tersedia. tertentu, pada planet tertentu, dan pada hari ke-T.
- **Langkah Neighbor (Swap):** Memilih 2 misi secara acak dari state saat ini, lalu menukar alokasi hari atau armada di antara kedua misi tersebut.
- **Objective Function (Cost):** Menghitung total pelanggaran dari *Hard Constraint* pada state saat ini.
- **Alasan Pemilihan Fungsi Objektif:** Fungsi ini dipilih karena nilai *Cost* merepresentasikan seberapa jauh jadwal dari kondisi valid. Tujuan pencarian adalah meminimalkan fungsi ini hingga mencapai angka 0 (Goal State tercapai, tidak ada konflik).

Algoritma Wajib dan Bonus Implementasi

- **Hill-Climbing (Bonus Terintegrasi):** Pemain dapat memilih seluruh varian Hill-Climbing, mencakup Basic (Steepest-Ascent), Sideways Move, Stochastic, dan Random Restart.
- **Simulated Annealing:** Pencarian yang mengizinkan pergerakan ke state yang lebih buruk (*worse move*) berdasarkan probabilitas temperatur yang berangsur menurun.
- **Genetic Algorithm:** Pencarian berbasis populasi yang menerapkan mekanisme *selection*, *crossover* silang jadwal, dan *mutation* acak.
- **Visualisasi Pencarian (Bonus Terintegrasi):** Program wajib menampilkan animasi perubahan state antar iterasi secara *real-time* di CLI (misalnya menggunakan *clear screen* dan *delay*) serta mencetak riwayat pergerakan nilai objektif selama pencarian berlangsung.

Daftar Fungsi Utama

- **generate_initial_state():** Membangkitkan list jadwal misi secara acak sebagai titik awal pencarian.
- **get_neighbors(state):** Mengembalikan state tetangga baru menggunakan mekanisme *Shift* atau *Swap*.
- **calculate_cost(state):** Mengkalkulasi nilai pelanggaran atau konflik dari sebuah state jadwal.
- **run_algorithm(type):** Menjalankan *loop* pencarian berdasarkan algoritma yang dipilih pengguna beserta animasinya.
- **print_visualization():** Menangani pembaruan layar CLI untuk menampilkan animasi pencarian secara interaktif.

Contoh Input dan Output

1. Inisialisasi dan Pemilihan Algoritma

```
=== INISIALISASI ASTRO-SCHEDULER ===
Mengatur 5 Misi, 3 Armada, 3 Planet, 3 Hari...
>> Initial State ter-generate (acak)!
>> Cost Awal: 4 (Terdapat 4 konflik jadwal)

Pilih Algoritma:
1. Hill-Climbing
2. Simulated Annealing
3. Genetic Algorithm
Masukkan pilihan (1-3): 1
```

```
Pilih Varian Hill-Climbing:  
a. Basic (Steepest-Ascent)  
b. Sideways Move  
c. Stochastic  
d. Random Restart  
Masukkan pilihan (a-d): d
```

2. Visualisasi Animasi Pencarian

```
[ANIMASI PENCARIAN BERJALAN...]  
Iterasi: 12 | Algoritma: Random Restart HC | Cost Saat Ini:  
1  
  
Jadwal Terkini:  
[Misi 0] Armada A -> Planet X (Hari 1)  
[Misi 1] Armada B -> Planet Y (Hari 1)  
[Misi 2] Armada A -> Planet Z (Hari 2)  
[Misi 3] Armada C -> Planet X (Hari 2)  
[Misi 4] Armada B -> Planet X (Hari 2)  <-- KONFLIK: Planet  
X dikunjungi Armada C & B di Hari 2!  
  
>> Action: Swap Misi 2 dan Misi 4...
```

3. Output Akhir

```
>> PENCARIAN SELESAI <<  
Alasan: Cost 0 tercapai pada iterasi ke-27 (Setelah 1x  
Restart)!  
  
--- JADWAL AKHIR VALID ---  
[Misi 0] Armada A -> Planet X (Hari 1)  
[Misi 1] Armada B -> Planet Y (Hari 1)  
[Misi 2] Armada B -> Planet Z (Hari 2)  
[Misi 3] Armada C -> Planet X (Hari 2)  
[Misi 4] Armada A -> Planet X (Hari 3)  
>> Seluruh misi berhasil dijadwalkan tanpa konflik!
```

Laporan Eksperimen: Prediksi Status Pinjaman (Logistic Regression)

Pendahuluan

Eksperimen ini berfokus pada pembangunan model klasifikasi untuk memprediksi probabilitas persetujuan pinjaman (`loan_status`). Sesuai dengan batasan yang diberikan, model dibangun sepenuhnya dari awal menggunakan pustaka `numpy` tanpa mengandalkan algoritma instan dari pustaka tingkat lanjut. Untuk memastikan pengujian yang objektif dan hasil yang dapat direproduksi, pengacakan data dikunci menggunakan seed (42), dan dataset dibagi menjadi 85% data latih serta 15% data validasi.

Metodologi Pra-pemrosesan dan Rekayasa Fitur

Karena Logistic Regression adalah algoritma linear, langkah rekayasa fitur dilakukan untuk membantu model menangkap pola non-linear pada dataset finansial ini. Proses pra-pemrosesan mencakup langkah-langkah berikut:

- **Encoding Kategorikal:** Fitur biner seperti jenis kelamin dan riwayat gagal bayar diubah menjadi nilai 0 dan 1. Fitur kepemilikan rumah (`RENT`, `OWN`, `MORTGAGE`, `OTHER`) dipecah menggunakan metode One-Hot Encoding manual.
- **Transformasi Non-Linear:** Untuk meratakan distribusi fitur yang memiliki rentang nilai ekstrem (seperti pendapatan), diterapkan transformasi logaritmik (`np.log1p`) pada kolom `person_income`.
- **Fitur Polinomial & Interaksi:** Menambahkan kolom fitur kuadrat untuk persentase pinjaman dan suku bunga guna menangkap efek eksponensial. Selain itu, dibuat

fitur interaksi risiko yang mengalikan `loan_percent_income` dengan `loan_int_rate`, serta fitur rasio pinjaman terhadap pendapatan (`income_loan_ratio`).

- Standardisasi (Z-Score): Seluruh matriks fitur diskalakan agar memiliki rata-rata 0 dan standar deviasi 1. Perhitungan rata-rata dan standar deviasi diambil murni dari data latih untuk mencegah kebocoran data (data leakage).
- Penambahan Intercept: Kolom matriks berisi angka 1 ditambahkan di awal dataset sebagai bias komputasi model.

Implementasi *Stable Logistic Regression*

Model `StableLogisticRegression` dibangun untuk menangani tantangan utama dataset, yaitu ketidakseimbangan kelas (class imbalance) dan risiko overfitting.

- Optimalisasi Bobot (Class Weights): Diterapkan pembobotan asimetris langsung ke dalam perhitungan fungsi kerugian (error). Kelas mayoritas (0/Ditolak) diberi bobot standar 1.0, sedangkan kelas minoritas (1/Disetujui) diberikan bobot penalti yang lebih berat, yaitu 2.2. Hal ini memaksa model untuk lebih memperhatikan kesalahan saat memprediksi kelas minoritas.
- L2 Regularization (Ridge): Untuk mencegah nilai bobot (θ) meledak akibat penambahan fitur polinomial, ditambahkan suku penalti L2 dengan parameter `lambda_reg` sebesar 0.00005. Penalti ini dihitung pada tahap pembaruan gradient descent, dengan mengecualikan bobot intercept dari hukuman regularisasi.
- Konfigurasi Pelatihan: Model dilatih menggunakan learning rate sebesar 0.8 dengan total 25.000 iterasi untuk mencapai konvergensi yang stabil pada tahap validasi. Fungsi sigmoid juga dilengkapi dengan kliping

batas (hingga rentang -250 dan 250) untuk mencegah overflow pada komputasi eksponensial.

Evaluasi dan Pencarian Threshold Optimal

Metrik evaluasi yang diimplementasikan secara manual adalah Macro F1-Score, yang menghitung keseimbangan antara precision dan recall untuk setiap kelas secara independen.

- Alih-alih menggunakan titik potong probabilitas baku (0.5), dilakukan iterasi dinamis dari angka 0.1 hingga 0.9 dengan interval 0.01 terhadap probabilitas prediksi data validasi.
- Berdasarkan proses iterasi tersebut, threshold optimal yang menghasilkan skor terbaik ditemukan pada angka 0.49.
- Dengan threshold tersebut, performa Stable Logistic Regression mencetak Macro F1-Score sebesar 0.8513 pada set validasi lokal.

Submisi Akhir

Setelah parameter dan threshold terbaik ditemukan, model final dilatih ulang menggunakan penggabungan seluruh dataset (train.csv) untuk memaksimalkan jumlah data pembelajaran. Iterasi pelatihan dinaikkan menjadi 30.000 iterasi untuk memastikan model konvergen sempurna pada dataset yang lebih besar. Hasil probabilitas dari data uji (test.csv) kemudian diputus menggunakan threshold 0.49 dan diekspor ke dalam file submission_final_stable.csv.